

LAPORAN TUGAS
DEADLOCK



Dosen Pengampu : Triawan Adi Cahyanto, M.K

Disusun Oleh:

Muhammad maulana wiramatajaya 2410651006

PROGRAM STUDY TEKNIK INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS MUHAMMADIYAH JEMBER

2024

1. Analisis Deadlock

1.1. Penyebab Terjadinya Deadlock

Deadlock terjadi karena:

- Thread 1 memegang lock1, lalu menunggu lock2
- Thread 2 memegang lock2, lalu menunggu lock1

Tidak ada thread yang dapat melanjutkan sehingga program hang selamanya.

1.2. Kondisi Deadlock

Kondisi Deadlock, Penjelasan dalam Program

1. Pengecualian Timbal Balik (Mutual Exclusion)

hanya dapat dipegang oleh satu thread pada satu waktu. Jika satu thread memegangnya, thread lain yang mencoba mengunci harus menunggu."

2. Pegang dan Tunggu (Hold and Wait)

Thread 1 memegang lock1 dan menunggu lock2. Thread 2 memegang lock2 dan menunggu lock1. Kedua thread memegang setidaknya satu sumber daya sambil menunggu sumber daya tambahan.

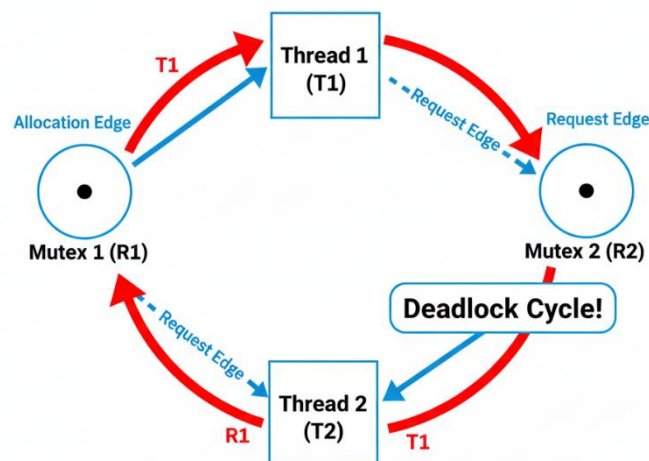
3. Tidak Ada Pra-emption (No Preemption)

Mutex tidak dapat diambil secara paksa (preempted) dari thread yang memegangnya. Thread hanya dapat unlock mutex secara sukarela setelah selesai menggunakannya.

4. Tunggu Melingkar (Circular Wait)

Terjadi rantai melingkar di mana Thread 1 menunggu sumber daya yang dipegang oleh Thread 2, dan Thread 2 menunggu sumber daya yang dipegang oleh Thread 1."

1.3. Diagram Resource Allocation



2. Impementasi Penegahan

2.1. Lock Ordering

2.1.1. Kode Program

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>
#include <stdlib.h> // Untuk rand() dan srand()
#include <time.h>   // Untuk time()
pthread_mutex_t lock1 = PTHREAD_MUTEX_INITIALIZER; // Deklarasi dua mutex
pthread_mutex_t lock2 = PTHREAD_MUTEX_INITIALIZER; // Deklarasi dua mutex

void acquire_resources(pthread_mutex_t* first, pthread_mutex_t* second, int id) {
    int got_first = 0;

    while (1) {
        if (pthread_mutex_trylock(first) == 0) {
            got_first = 1;
            printf("Thread %d: Berhasil dapat Lock A (Pertama)!\n", id);
            if (pthread_mutex_trylock(second) == 0) {
                printf("Thread %d: Berhasil dapat Lock B (Kedua)!\n", id);
                return;
            } else {
                printf("Thread %d: GAGAL dapat Lock B. Melepaskan Lock A dan mencoba lagi.\n", id);

                pthread_mutex_unlock(first);
                got_first = 0; // Reset status

                usleep(rand() % 100000 + 50000); // Tunggu 50ms - 150ms
            }
        } else {
            usleep(rand() % 100000 + 50000); // Tunggu 50ms - 150ms
        }
    }
}

void* thread1_func(void* arg) { // Fungsi untuk Thread 1
    int id = 1;
    printf("Thread %d: Mencoba mengakuisisi Lock 1 dan Lock 2.\n", id);

    acquire_resources(&lock1, &lock2, id);
    printf("Thread %d: Selesai bekerja...\n", id);
    sleep(1); // Simulasi kerja
    pthread_mutex_unlock(&lock2);
    pthread_mutex_unlock(&lock1);

    return NULL;
}

void* thread2_func(void* arg) {
    int id = 2;
    printf("Thread %d: Mencoba mengakuisisi Lock 2 dan Lock 1.\n", id);
    acquire_resources(&lock2, &lock1, id);
    printf("Thread %d: Selesai bekerja...\n", id);
    sleep(1); // Simulasi kerja
    pthread_mutex_unlock(&lock1);
    pthread_mutex_unlock(&lock2);

    return NULL;
}

int main() {
    srand(time(NULL)); // Inisialisasi generator angka acak
    pthread_t thread1, thread2;
    printf("=== SIMULASI PENCEGAHAN DEADLOCK: TRY-LOCK DENGAN RETRY ===\n");
    pthread_create(&thread1, NULL, thread1_func, NULL);
    pthread_create(&thread2, NULL, thread2_func, NULL);
    pthread_join(thread1, NULL);
    pthread_join(thread2, NULL);
    printf("=== PROGRAM SELESAI DENGAN AMAN ===\n");
}
```

2.1.2. Output

```
wiramatajaya@LAPTOP-7B38G7J9:~/pertemuan_6$ ./modif
=== SIMULASI PENCEGAHAN DEADLOCK: TRY-LOCK DENGAN RETRY ===
Thread 1: Mencoba mengakuisisi Lock 1 dan Lock 2.
Thread 1: Berhasil dapat Lock A (Pertama)!
Thread 1: Berhasil dapat Lock B (Kedua)!
Thread 1: Selesai bekerja...
Thread 2: Mencoba mengakuisisi Lock 2 dan Lock 1.
Thread 2: Berhasil dapat Lock A (Pertama)!
Thread 2: Berhasil dapat Lock B (Kedua)!
Thread 2: Selesai bekerja...
=== PROGRAM SELESAI DENGAN AMAN ===
wiramatajaya@LAPTOP-7B38G7J9:~/pertemuan_6$ |
```

2.2. Try Lock

2.2.1. Kode Program

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>
#include <stdlib.h> // Untuk rand() dan srand()
#include <time.h>   // Untuk time()

// Deklarasi dua mutex
pthread_mutex_t lock1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t lock2 = PTHREAD_MUTEX_INITIALIZER;

void acquire_resources(pthread_mutex_t* first, pthread_mutex_t* second, int id) {
    int got_first = 0;

    while (1) {
        if (pthread_mutex_trylock(first) == 0) {
            got_first = 1;
            printf("Thread %d: Berhasil dapat Lock A (Pertama)!\n", id);

            if (pthread_mutex_trylock(second) == 0) {
                printf("Thread %d: Berhasil dapat Lock B (Kedua)!\n", id);
                return;
            } else {
                printf("Thread %d: GAGAL dapat Lock B. Melepaskan Lock A dan mencoba lagi.\n", id);

                pthread_mutex_unlock(first);
                got_first = 0; // Reset status

                usleep(rand() % 100000 + 50000); // Tunggu 50ms - 150ms
            }
        } else {
            usleep(rand() % 100000 + 50000); // Tunggu 50ms - 150ms
        }
    }
}

// Fungsi untuk Thread 1
void* thread1_func(void* arg) {
    int id = 1;
    printf("Thread %d: Mencoba mengakuisisi Lock 1 dan Lock 2.\n", id);

    acquire_resources(&lock1, &lock2, id);

    printf("Thread %d: Selesai bekerja...\n", id);
    sleep(1); // Simulasi kerja

    pthread_mutex_unlock(&lock2);
    pthread_mutex_unlock(&lock1);

    return NULL;
}

void* thread2_func(void* arg) {
    int id = 2;
    printf("Thread %d: Mencoba mengakuisisi Lock 2 dan Lock 1.\n", id);

    acquire_resources(&lock2, &lock1, id);

    printf("Thread %d: Selesai bekerja...\n", id);
    sleep(1); // Simulasi kerja

    pthread_mutex_unlock(&lock1);
    pthread_mutex_unlock(&lock2);

    return NULL;
}

int main() {
    srand(time(NULL)); // Inisialisasi generator angka acak
    pthread_t thread1, thread2;
    printf("=== SIMULASI PENCEGAHAN DEADLOCK: TRY-LOCK DENGAN RETRY ===\n");

    // Membuat dua thread
    pthread_create(&thread1, NULL, thread1_func, NULL);
    pthread_create(&thread2, NULL, thread2_func, NULL);

    pthread_join(thread1, NULL);
    pthread_join(thread2, NULL);

    printf("=== PROGRAM SELESAI DENGAN AMAN ===\n");
    return 0;
}
```

2.2.2. Output

```

=== PENCEGAHAN DEADLOCK: TRY-LOCK & TIMED-LOCK ===
Thread 2: Mulai berjalan.
Thread 2: Mencoba mengambil **Lock 2**...
Thread 2: **Lock 2** berhasil diambil. Mencoba mengambil **Lock 1** (timed-lock)...
Thread 2: ✓ Berhasil dapat kedua lock (L2 & L1)!
Thread 1: Mulai berjalan.
Thread 1: Mencoba mengambil **Lock 1**...
Thread 2: Melepas **Lock 1** dan **Lock 2**..
Thread 2: Selesai (retry: 0 kali)
Thread 1: **Lock 1** berhasil diambil. Mencoba mengambil **Lock 2** (try-lock)...
Thread 1: ✓ Berhasil dapat kedua lock (L1 & L2)!
Thread 1: Melepas **Lock 2** dan **Lock 1**..
Thread 1: Selesai (retry: 0 kali)

=== PROGRAM SELESAI TANPA DEADLOCK ===
wiramatajaya@LAPTOP-7B38G7J9:~/pertemuan_6$

```

2.3. Perbandingan

Fitur	Lock Ordering	Try-lock
Kondisi yang Dicegah	Tunggu Melingkar (<i>Circular Wait</i>).	Pegang dan Tunggu (<i>Hold and Wait</i>).
Kompleksitas Implementasi	Relatif sederhana jika jumlah <i>lock</i> kecil. Membutuhkan dokumentasi dan penegakan urutan yang ketat.	Lebih kompleks. Membutuhkan <i>loop while(1)</i> , <i>trylock()</i> , <i>unlock()</i> , dan mekanisme <i>sleep/retry</i> (<i>backoff</i>).
Kinerja (Kelebihan)	Efisien. Ketika <i>thread</i> mendapatkan <i>lock</i> pertama, ia langsung menunggu <i>lock</i> kedua tanpa <i>overhead retry</i> .	<i>Thread</i> tidak akan pernah <i>stuck</i> menunggu; sumber daya yang dipegang akan segera dibebaskan jika gagal.
Kinerja (Kekurangan)	<i>Thread</i> mungkin harus menunggu lama dalam <i>pthread_mutex_lock()</i> jika <i>lock</i> pertama sedang dipegang.	Berpotensi mengalami kelaparan (<i>starvation</i>) jika <i>thread</i> terus-menerus gagal dan <i>retry</i> tanpa pernah berhasil. Ada <i>overhead</i> CPU dari <i>retry</i> dan <i>unlock</i> berulang.
Skalabilitas	Sulit diskalakan. Urutan global harus ditetapkan untuk ratusan <i>lock</i> , yang bisa sulit dipertahankan dan rentan kesalahan.	Lebih mudah diskalakan. Tidak memerlukan urutan global yang ketat, namun kompleksitas kodenya meningkat.
Jaminan Pencegahan	Memberikan jaminan mutlak mencegah <i>deadlock</i> selama urutan penguncian ditaati.	Mencegah <i>deadlock</i> tetapi dapat menyebabkan masalah kelaparan dan penurunan kinerja karena <i>retry</i> yang berlebihan.

3. Banker's Algorithm

3.1. Kode Program

```
#include <stdio.h>
#include <stdbool.h>

// Jumlah Proses (n) dan Sumber Daya (m)
#define N 4 // 4 Proses (P0, P1, P2, P3)
#define M 3 // 3 Sumber Daya (A, B, C)

// Data Banker's Algorithm
int Available[M] = {5, 4, 3}; // Tersedia: [A, B, C]
int Allocation[N][M] = {
    {1, 1, 0}, // P0
    {2, 0, 0}, // P1
    {0, 0, 1}, // P2
    {1, 0, 1}  // P3
};

int Max[N][M] = {
    {5, 2, 2}, // P0
    {3, 3, 3}, // P1
    {4, 0, 1}, // P2
    {1, 4, 3}  // P3
};

int Need[N][M]; // Akan dihitung

// Fungsi untuk menghitung matriks Need
void calculateNeed() {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < M; j++) {
            Need[i][j] = Max[i][j] - Allocation[i][j];
        }
    }
}

// Fungsi Safety Check (Inti dari Banker's Algorithm)
bool isSafe(int current_available[], int safe_sequence[]) {
    // 1. Inisialisasi Work dan Finish
    int Work[M];
    for (int j = 0; j < M; j++) {
        Work[j] = current_available[j];
    }
}
```

Safety Check (Inti dari Banker's Algorithm)

```
bool isSafe(int current_available[], int safe_sequence[]) {  
    // 1. Inisialisasi Work dan Finish  
    int Work[M];  
    for (int j = 0; j < M; j++) {  
        Work[j] = current_available[j];  
    }  
    bool Finish[N] = {false};  
    int count = 0;  
  
    // Ulangi langkah sampai semua proses selesai atau tidak ada perubahan  
    while (count < N) {  
        bool found = false;  
  
        for (int i = 0; i < N; i++) {  
            // Cek apakah proses i belum selesai DAN Need[i] <= Work  
            if (Finish[i] == false) {  
                // Cek kondisi Need[i] <= Work  
                bool can_satisfy = true;  
                for (int j = 0; j < M; j++) {  
                    if (Need[i][j] > Work[j]) {  
                        can_satisfy = false;  
                        break;  
                    }  
                }  
                if (can_satisfy) { // Jika Need bisa dipenuhi:  
                    // 2. Sumber daya dialokasikan ke proses i, dan Work diupdate  
                    for (int j = 0; j < M; j++) {  
                        Work[j] += Allocation[i][j];  
                    }  
                    Finish[i] = true;  
                    safe_sequence[count] = i; // Catat urutan aman  
                    count++;  
                    found = true;}  
                }  
            }  
  
            if (found == false) { // Jika tidak ada proses yang ditemukan dalam satu iterasi, sistem tidak aman  
                return false;} // Sistem tidak aman  
        }  
        return true; // Sistem aman  
    }  
}  
  
// Fungsi utama untuk menangani permintaan sumber daya  
void requestResource(int process_id, int Request[]) {  
    printf("\n--- Proses %d Meminta Sumber Daya ---\n", process_id);  
    printf("Permintaan: A=%d, B=%d, C=%d\n", Request[0], Request[1], Request[2]);  
  
    // 1. Cek apakah Request <= Need  
    for (int j = 0; j < M; j++) {  
        if (Request[j] > Need[process_id][j]) {
```

```

int main() {
    calculateNeed();

    printf("=== BANKER'S ALGORITHM SIMULATION (4 Proses, 3 Sumber Daya) ===\n");
    printf("Initial Available: A=%d, B=%d, C=%d\n", Available[0], Available[1], Available[2]);

    // A. Cek Keamanan Awal Sistem
    int initial_safe_sequence[N];
    printf("\n--- Cek Keamanan Awal ---\n");
    if (isSafe(Available, initial_safe_sequence)) {
        printf("STATUS AWAL: Sistem berada dalam keadaan AMAN.\n");
        printf("Urutan Aman Awal: <P%d", initial_safe_sequence[0]);
        for (int i = 1; i < N; i++) {
            printf(", P%d", initial_safe_sequence[i]);
        }
        printf(">\n");
    } else {
        printf("STATUS AWAL: Sistem berada dalam keadaan TIDAK AMAN.\n");
    }

    // B. Implementasi Request Resource (Skenario 1: Request yang Aman)
    int request1[M] = {1, 0, 0}; // P1 meminta 1 A, 0 B, 0 C
    requestResource(1, request1);

    // C. Implementasi Request Resource (Skenario 2: Request yang Berpotensi Tidak Aman)
    int request2[M] = {4, 0, 0}; // P2 meminta 4 A, 0 B, 0 C
    requestResource(2, request2);
    printf("\n===== \n");

    return 0;
}

```

3.2. Output

```

wiramatajaya@LAPTOP-7B38G7J9:~/pertemuan_6$ gcc -o banker banker.c
wiramatajaya@LAPTOP-7B38G7J9:~/pertemuan_6$ ./banker
=== BANKER'S ALGORITHM SIMULATION (4 Proses, 3 Sumber Daya) ===
Initial Available: A=5, B=4, C=3

--- Cek Keamanan Awal ---
STATUS AWAL: Sistem berada dalam keadaan AMAN.
Urutan Aman Awal: <P0, P1, P2, P3>

--- Proses 1 Meminta Sumber Daya ---
Permintaan: A=1, B=0, C=0
>> KONDISI AMAN: Permintaan diizinkan. Sistem tetap berada dalam keadaan aman.
Urutan Aman yang ditemukan: <P0, P1, P2, P3>

--- Proses 2 Meminta Sumber Daya ---
Permintaan: A=4, B=0, C=0
>> KONDISI AMAN: Permintaan diizinkan. Sistem tetap berada dalam keadaan aman.
Urutan Aman yang ditemukan: <P1, P3, P0, P2>

=====
wiramatajaya@LAPTOP-7B38G7J9:~/pertemuan_6$

```

4. Real Case

4.1. Kode Program

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>
#include <time.h>
pthread_mutex_t lock1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t lock2 = PTHREAD_MUTEX_INITIALIZER;
int deadlock_detected = 0;
void* thread1_func(void* arg) {
    struct timespec timeout;
    int result;
    printf("Thread 1: Mengambil Lock 1\n");
    pthread_mutex_lock(&lock1);
    sleep(1);
    printf("Thread 1: Mencoba mengambil Lock 2 (dengan timeout 2 detik)...\n");
    // Set timeout 2 detik
    clock_gettime(CLOCK_REALTIME, &timeout);
    timeout.tv_sec += 2;
    result = pthread_mutex_timedlock(&lock2, &timeout);
    if (result != 0) {
        printf("Thread 1: TIMEOUT! Kemungkinan terjadi deadlock.\n");
        deadlock_detected = 1;
        pthread_mutex_unlock(&lock1);
        return NULL;
    }
    printf("Thread 1: Berhasil dapat kedua lock\n");
    pthread_mutex_unlock(&lock2);
    pthread_mutex_unlock(&lock1);
    return NULL;
}
void* thread2_func(void* arg) {
    printf("Thread 2: Mengambil Lock 2\n");
    pthread_mutex_lock(&lock2);
    sleep(1);
    printf("Thread 2: Mencoba mengambil Lock 1...\n");
    pthread_mutex_lock(&lock1);
    printf("Thread 2: Berhasil dapat kedua lock\n");
    pthread_mutex_unlock(&lock1);
    pthread_mutex_unlock(&lock2);
    return NULL;
}
```

```

int main() {
pthread_t thread1, thread2;
printf("=== DETEKSI DEADLOCK DENGAN TIMEOUT ===\n\n");
pthread_create(&thread1, NULL, thread1_func, NULL);
pthread_create(&thread2, NULL, thread2_func, NULL);
pthread_join(thread1, NULL);
pthread_join(thread2, NULL);
if (deadlock_detected) {
printf("\n[SISTEM] Deadlock terdeteksi dan ditangani!\n");
} else {
printf("\n[SISTEM] Program berjalan normal.\n");
}
return 0;
}

```

4.2. Output Program

```

wiramatajaya@LAPTOP-7B38G7J9:~/pertemuan_6$ gcc -o safe safe_transfer.c -pthread
wiramatajaya@LAPTOP-7B38G7J9:~/pertemuan_6$ ./safe
=== SIMULASI PENCEGAHAN DEADLOCK (LOCK ORDERING) ===
Initial: R1: 500.00 | R2: 700.00. Transfer: 100.00
-----

Orang A (SAFE): Mulai transfer dari R1 ke R2. Order: Lock R1, lalu R2
Orang A (SAFE): Mencoba Lock Rekening 1 (FIRST)...
Orang A (SAFE): ✓ Lock R1 berhasil diambil.

Orang B (SAFE): Mulai transfer dari R2 ke R1. Order: Lock R1, lalu R2
Orang B (SAFE): Mencoba Lock Rekening 1 (FIRST)...
Orang A (SAFE): Mencoba Lock Rekening 2 (SECOND)...
Orang A (SAFE): ✓ Lock R2 berhasil diambil.
--- Orang A (SAFE): Transfer sukses! Saldo R1: 400.00, Saldo R2: 800.00 ---
Orang B (SAFE): ✓ Lock R1 berhasil diambil.
Orang B (SAFE): Mencoba Lock Rekening 2 (SECOND)...
Orang B (SAFE): ✓ Lock R2 berhasil diambil.
--- Orang B (SAFE): Transfer sukses! Saldo R2: 700.00, Saldo R1: 500.00 ---

=== PROGRAM SELESAI TANPA DEADLOCK ===
Final: R1: 500.00 | R2: 700.00
wiramatajaya@LAPTOP-7B38G7J9:~/pertemuan_6$

```

4.3. Analisis

Program ini mendeteksi deadlock menggunakan timeout. Jika lock tidak didapat dalam 2 detik, thread akan berhenti mencoba.